PaySphere Wallet Management System

1. Introduction

PaySphere Wallet Management System is a secure digital wallet platform that enables users to register, authenticate, manage wallet balances, perform money transfers, and maintain a complete transaction history. The solution is designed using a microservices architecture with an API Gateway to provide scalability, maintainability, and secure service separation.

2. Business Problem

Organizations require a secure backend platform to manage digital wallet operations while ensuring transaction integrity, authentication, auditability, and maintainable architecture.

3. Objectives

- Provide secure JWT-based authentication.
- Allow users to manage wallet balances and transfers.
- Maintain complete transaction history.
- Provide a maintainable layered architecture.
- Ensure logging, exception handling, and unit testing.

4. Scope

In Scope

- API Gateway
- Auth Microservice
- Wallet Microservice
- Role-based Authorization
- Wallet Operations
- Transaction History
- Swagger
- Serilog
- Unit Testing

Out of Scope

- Payment Gateway Integration
- Notifications
- Rewards Engine
- Docker
- RabbitMQ
- Mobile Application

5. User Roles

Admin: Manage users and monitor wallet operations.
User: Register, login, manage wallet, transfer money, and view transactions.

6. Functional Requirements

ID	Requirement	Description
FR-01	User Registration	Create a new account.
FR-02	Login	Authenticate and issue JWT.
FR-03	Create Wallet	Create wallet for user.
FR-04	Top-up	Add balance.
FR-05	Withdraw	Withdraw balance.
FR-06	Transfer	Transfer between wallets.
FR-07	History	View paginated transaction history.
FR-08	Roles	Admin/User authorization.

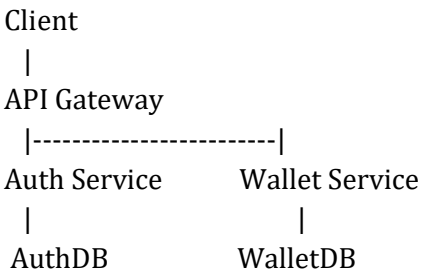
7. Business Rules

- Wallet balance cannot become negative.
- Transfer amount must be greater than zero.
- User cannot transfer to own wallet.
- Only authenticated users can perform wallet operations.
- Only administrators can manage users.

8. Non-Functional Requirements

- Security using JWT.
- Serilog logging.
- Global exception handling.
- Response time under normal load below 2 seconds.
- Maintainable layered architecture.
- Audit logging.

9. Solution Architecture



10. Microservices

Auth Service

- Register
- Login
- JWT
- Profile
- Role Management

Wallet Service

- Create Wallet
- Top-up
- Withdraw
- Transfer
- Balance
- Transactions

11. Technology Stack

- ASP.NET Core 8
- C#
- Entity Framework Core
- SQL Server
- Ocelot API Gateway
- Serilog
- Swagger
- NUnit
- Moq

12. Design Patterns

- Layered Architecture
- Repository Pattern
- Service Pattern
- Generic Repository
- Dependency Injection
- DTO Pattern

13. Security

- JWT Authentication
- Role-Based Authorization
- Password Hashing
- Input Validation

14. Logging Strategy

- Serilog
- Information, Warning, Error and Fatal logs
- File and Console logging
- Request/Response logging

15. Exception Strategy

- Global Exception Middleware
- Custom Exceptions: UserNotFoundException, DuplicateUserException, WalletNotFoundException, InsufficientBalanceException

16. Database

AuthDB
- Users
- Roles

WalletDB
- Wallets
- Transactions
- AuditLogs

17. REST APIs

Method	Endpoint
POST	/api/v1/auth/register
POST	/api/v1/auth/login
GET	/api/v1/auth/profile
POST	/api/v1/wallet/create
POST	/api/v1/wallet/topup
POST	/api/v1/wallet/withdraw
POST	/api/v1/wallet/transfer
GET	/api/v1/wallet/transactions

18. Unit Testing Strategy

- Framework: NUnit
- Mocking: Moq
- Test only Service Layer
- Mock repositories
- Cover success and exception scenarios

19. Future Enhancements

- RabbitMQ
- Rewards Engine
- Notifications
- Docker

- Redis Cache
- Payment Gateway Integration

20. Conclusion

PaySphere Wallet Management System is designed as a maintainable, secure, enterprise-style backend solution. The architecture emphasizes clean separation of concerns, robust security, structured logging, centralized exception handling, and comprehensive unit testing while remaining extensible for future enhancements.